

Typer av ärvning

Den typ av ärvning vi hittills använt är den normala `public` ärvningen. Typen av ärvning specificeras som tidigare nämnts då klassen definieras, t.ex.:

```
class Foo : public Bar {  
    ...  
};
```

Det finns dock andra typer av ärvning än enbart `public`. De två andra är `private` och `protected`. Typen av ärvning påverkar hur metoder och datamedlemmar är tillgängliga för subklassen samt externa entiteter som använder objekt av klassen.

`public` ärvning

Detta är den normala formen av ärvning. Alla metoder som är definierade som `public` i basklassen är `public` även för subklassen. Data som är deklarerat som `private` i basklassen kan inte accesseras av subklasser eller externa entiteter. I princip ändras inget från hur det är definierat i basklassen.

`private` ärvning

Denna typ av ärvning är den mest restriktiva formen av ärvning. Den gömmer alla detaljer av basklassen för externa entiteter. Om vi t.ex. ärver klassen `Circle` privat av klassen `Shape` enligt:

```
class Circle : private Shape {  
    ...  
};
```

då kan ingen som använder objekt av klassen `Circle` referera till metoden `origin()`, eftersom det nu är en privat metod i `Circle`. Privat ärvning kan användas som en form av *has-a* förhållande. Eftersom ingen kan referera till basklassen är det i princip samma sak som om klassen hade haft en dylik datamedlem. Denna typ av ärvning är den minst använda.

`protected` ärvning

Detta är en relativt speciell typ av ärvning. Den är designad för att göra ärvning lätt genom att tillåta basklasser att accessera vissa metoder och medlemmar, medan externa entiteter inte tillåts det. Om vi t.ex. ser på vår `Shape` klass igen märker vi säkert förr eller senare att medlemmen `m-Origin` som innehåller positionen för figuren används ofta av subklasser. Dessa måste varje gång anropa metoderna `origin()` och `setOrigin()` för att accessera denna information. Det vore praktiskt om subklasser istället direkt kunde manipulera `m-Origin` på något sätt. Ett alternativ är att göra den en `public` medlem och sedan ärva `Shape` `public`, men då kan vem som helst manipulera `m-Origin`, vilket inte är bra. Om vi istället gör medlemmen `protected` kan vi använda oss av ett alternativt och bättre sätt. Vi kan då skriva `Shape` på detta sätt:

```
class Shape {  
public:
```

```
// konstruktor
Shape (const Coordinate & Origin);

// virtuella metoder för area och omkrets
virtual float area () const;
virtual float circum () const;

// position för figuren
const Coordinate & origin () const;
void setOrigin (const Coordinate & Origin);

protected:
    // position
    Coordinate m-Origin;
};
```

Enda skillnaden är flaggan `protected`. I praktiken kan subclasser nu accessera `m-Origin` direkt utan att använda de två accessmetoderna, medan externa klasser och funktioner ännu är begränsade till att använda accessmetoderna. Alla subclasser av subclasser o.s.v. kan accessera `m-Origin` så länge som ärvningen hela tiden är `public` eller `protected`. Man kan alltså även *ärva* `protected` förutom att ha dataskyddsnivån `protected`. Det kan verka en aning klurigt, och det är det tyvärr också.

Om man ärver en klass som `protected` blir alla metoder och all data som varit `public` till `protected`, medan all data som varit `protected` i basklassen förblir det i subclassen. Man använder skyddat arv för att införa begränsingar på vad utomstående kan accessera, medan subclasser ännu har full tillgång till allt som varit skyddat som `public` eller `protected` i basklassen. För en utomstående är alltså `protected` arv detsamma som `private`. Det krävs en aning tankearbete innan man fullt förstår hur `protected` arv och dataskydd fungerar.

Man kan göra en basklass *abstrakt* (såsom abstrakta basklasser med virtuella metoder) genom att göra dess konstruktor `protected`. På så vis kan ingen utomstående accessera konstruktorn, och således ej heller instantiera klassen. Subklasser däremot har full tillgång till konstruktorn och kan anropa den. På så vis kan man instantiera subclasser.

Sammanfattning

Följande tabell sammanfattar hur de olika skyddsnivåerna påverkar de olika arvstyperna.

Tabell 15-1. Skyddsnivåer och arvstyper

	public ärvning	protected ärvning	private ärvning
public medlemmar blir	public medlemmar i ärvda klassen	protected medlemmar i ärvda klassen	private medlemmar i ärvda klassen
protected medlemmar blir	protected medlemmar i ärvda klassen	protected medlemmar i ärvda klassen	private medlemmar i ärvda klassen
private medlemmar blir	ej tillgängliga i ärvda klassen	ej tillgängliga i ärvda klassen	ej tillgängliga i ärvda klassen

[Förutgående](#)

Överlagring av metoder

[Hem](#)

[Upp](#)

[Nästa](#)

Referera till basklasser